

Translated

From: e-maxx.ru

Finding the nearest pair of points

Problem statement

Given n points on the plane. Each point p_i is defined by its coordinates (x_i, y_i) . It is required to find among them two such points, such that the distance between them is minimal:

$$\min_{\substack{i, j=0 \dots n-1, \\ i \neq j}} \rho(p_i, p_j).$$

We take the usual Euclidean distances:

$$\rho(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

The trivial algorithm - iterating over all pairs and calculating the distance for each — works in $O(n^2)$.

The algorithm running in time $O(n \log n)$ is described below. This algorithm was proposed by Shamos and Hoey in 1975. (Source: Ch. 5 Notes of *Algorithm Design* by Kleinberg & Tardos, also see [here](#)) Preparata and Shamos also showed that this algorithm is optimal in the decision tree model.

Algorithm

We construct an algorithm according to the general scheme of **divide-and-conquer** algorithms: the algorithm is designed as a recursive function, to which we pass a set of points; this recursive function splits this set in half, calls itself recursively on each half, and then performs some operations to combine the answers. The operation of combining consists of detecting the cases when one point of the optimal solution fell into one half, and the other point into the other (in this case, recursive calls from each of the halves cannot detect this pair separately). The main difficulty, as always in case of divide and conquer algorithms, lies in the effective implementation of the merging stage. If a set of n points is passed to the recursive function, then the merge stage should work no more than $O(n)$, then the asymptotics of the whole algorithm $T(n)$ will be found from the equation:

$$T(n) = 2T(n/2) + O(n).$$

The solution to this equation, as is known, is $T(n) = O(n \log n)$.